

Técnicas de aprendizaje maquina sobre dataset del ICFES

Angel David Gomez Pastrana
Leidy Yulliana Quiñones Gonzalez

Resumen— Este documento busca plasmar los resultados en la comparación de métodos de aprendizaje maquina implementado a un dataset; tres de ellos para la clasificación de datos.

Abstract— This document seeks to present the results of comparing machine learning methods implemented on a dataset; three of them for data classification.

Index Terms—Machine learning, Classification, dataset, Random Forest, CatBoost, k-NN, comparison, analysis.

I. Introducción

Desde una época temprana en la historia, la humanidad ha llevado registros de varios fenómenos con la intención de clasificarlos por categorías, número de ocurrencias o cualquier otra variable que pudiese ser el límite entre un conjunto de datos y otro; En Colombia, hubo la necesidad de tener una clasificación de su población mediante su educación, para esto, en 1966, la Asociación de Universidades y el Fondo Universitario crearon un examen estandarizado el cual sería obligatorio para todos los estudiantes de grado undécimo; en éste se evaluaban conocimientos en diferentes áreas como biología, matemáticas, español, y entre otras.

Con el tiempo, este examen evolucionó hasta convertirse en lo que hoy se conoce como las pruebas Saber 11, administradas por el Instituto Colombiano para la Evaluación de la Educación (ICFES). Estas pruebas no sólo permiten evaluar el desempeño académico individual de los estudiantes, sino que también generan un volumen considerable de datos demográficos, socioeconómicos y pedagógicos.

El análisis de estos datos resulta fundamental para identificar patrones, brechas educativas y factores asociados al rendimiento estudiantil. No obstante, la gran cantidad de registros y la multidimensionalidad de las variables dificultan su interpretación mediante métodos estadísticos tradicionales. En este contexto, las técnicas de aprendizaje automático (machine learning)

se presentan como una herramienta poderosa para abordar problemas de clasificación, predicción y descubrimiento de conocimiento en grandes conjuntos de datos.

El presente trabajo tiene como objetivo aplicar y comparar diferentes algoritmos de clasificación supervisada sobre un conjunto de datos reales de las pruebas Saber 11, con el fin de predecir el desempeño de los estudiantes en categorías predefinidas. Se exploran técnicas como CatBoost, bosques aleatorios (random forest) y XG Boost (Extreme Gradient Boost), evaluando su precisión, sensibilidad y especificidad. De esta manera, se busca no sólo contribuir al análisis educativo en Colombia, sino también demostrar la aplicabilidad del aprendizaje automático en contextos de evaluación a gran escala.

II. Desarrollo de Contenidos

Para entender mejor los resultados que mostraremos, se requiere ciertos conocimientos previos que desarrollaremos en esta sección, es claro que hay conocimientos que se requieren pero no se abordarán explícitamente porque se asume que el lector los tiene, estos son manejo de conceptos de álgebra lineal, cálculo vectorial y manejo de matemáticas básicas.

A. Machine Learning(Aprendizaje automático):

A.1 Definición y alcance:

El aprendizaje automático (machine learning, ML) es un subcampo de la inteligencia artificial que dota a los sistemas computacionales de la capacidad de aprender patrones a partir de datos empíricos, sin necesidad de ser programados explícitamente con reglas lógicas o matemáticas fijas para cada situación [1]. En esencia, un modelo de ML construye una función de mapeo $f: X \rightarrow Y$ que transforma un vector de características de entrada $x \in X$ (por ejemplo, mediciones de sensores, atributos de un objeto) en una salida $y \in Y$ (una clase, un valor numérico, etc.), optimizando sus parámetros a partir de ejemplos.

A.2 Tipos de aprendizaje relevantes

Si bien existen múltiples paradigmas en ML (supervisado, no supervisado, semisupervisado, por refuerzo), el presente estudio se limita al aprendizaje supervisado (supervised learning), caracterizado por:

- Un conjunto de entrenamiento
 $D_{train} = \{(x_i, y_i)\}_{i=1}^N$, donde cada x_i es un vector de d características y y_i es una etiqueta de clase predefinida.
- El objetivo es aprender un clasificador $h(x)$ que minimice el error de clasificación sobre datos no vistos.

En el aprendizaje supervisado se distinguen dos subáreas:

- Regresión: la variable de salida es continua (ej. predecir temperatura).
- Clasificación: la variable de salida es discreta (ej. "normal" vs. "anomalía"). Este es el enfoque de este informe.

A.3 Flujo de trabajo típico para clasificación

Para que los resultados experimentales sean comparables y reproducibles, todo estudio de ML sigue un flujo metodológico estándar, cuyas etapas se resumen a continuación:

1. Adquisición y preparación de datos:
 - Recolección de ejemplos representativos del dominio.
 - Limpieza (valores nulos, ruido, outliers).
 - Codificación de variables categóricas, escalado o normalización (según requerimientos de cada técnica).
2. División del conjunto de datos:
 - Entrenamiento (típicamente 60-80% de los datos): usado para ajustar los parámetros del modelo.
 - Validación (10-20%): usado para ajustar hiperparámetros y evitar sobreajuste.
 - Prueba (10-20%): usado únicamente al final para estimar el desempeño final en datos no vistos.
 - Métodos como k-fold cross-validation (ej. 10 particiones) permiten un uso más eficiente de los datos cuando el tamaño es limitado.
3. Selección del algoritmo: Elección de una técnica específica (ej. árboles de decisión, SVM, redes neuronales). Cada una impone distintos

supuestos sobre la distribución de los datos (linealidad, independencia, etc.).

4. Entrenamiento: El algoritmo optimiza una función de costo $J(\theta)$ sobre D_{train} , ajustando sus parámetros internos θ . Por ejemplo, en regresión logística se minimiza la entropía cruzada; en SVM se maximiza el margen.
5. Evaluación y métricas: Sobre el conjunto de pruebas se calculan indicadores cuantitativos que permiten comparar técnicas. Para clasificación binaria o multiclase se utilizan:
 - Exactitud (Accuracy): $\frac{VP+VN}{VP+VN+FP+FN}$. Proporción de aciertos totales. Útil cuando las clases están balanceadas.
 - Puntaje F1: Media armónica entre precisión y exhaustividad:
 $2 * \frac{Precisión * Recall}{Precisión + Recall}$. Especialmente útil con clases desbalanceadas.
 - Matriz de confusión: tabla que muestra VP, VN, FP, FN por cada clase.
 - Curva ROC y AUC: evalúa la capacidad de discriminación del clasificador a distintos umbrales.
6. Análisis de resultados: Se comparan las métricas entre las tres técnicas, identificando cuál ofrece mejor generalización, menor varianza y mayor robustez ante ruido o desbalance de clases.

A.4 Conceptos fundamentales de error: sesgo, varianza y ruido

Para interpretar por qué una técnica de clasificación puede superar a otra, es necesario comprender la descomposición del error esperado de generalización [2]:

$$Error(h) = Sesgo^2 + Varianza + Ruido$$

- Sesgo (bias): Error sistemático introducido por supuestos demasiado rígidos del modelo. Un clasificador con alto sesgo (ej. regresión lineal sobre datos no lineales) no puede aprender la verdadera frontera de decisión → underfitting. Síntomas: error alto tanto en entrenamiento como en prueba.
- Varianza (variance): Sensibilidad del modelo a pequeñas fluctuaciones en los datos de entrenamiento. Un clasificador con alta varianza (ej. árbol de decisión sin podar) memoriza ruido y detalles irrelevantes → overfitting. Síntomas: error muy bajo en entrenamiento pero alto en prueba.
- Ruido (irreducible error): Varianza inherente a los datos o errores de etiquetado. Ningún

modelo puede reducirlo.

El trade-off sesgo-varianza implica que aumentar la complejidad del modelo reduce el sesgo pero aumenta la varianza. La mejor técnica para un problema dado es la que logra el equilibrio óptimo para el tamaño y naturaleza del conjunto de datos disponible.

Entendiendo el concepto de Machine Learning, vamos a enfocarnos en las técnicas que usaremos para la clasificación.

B. Random Forest:

B.1 Definición y origen

Random Forest es un algoritmo de aprendizaje supervisado propuesto por Leo Breiman [1] que pertenece a la familia de métodos de ensemble learning (aprendizaje por conjuntos). Como su nombre indica, construye un conjunto (forest) de múltiples árboles de decisión (decision trees) durante la fase de entrenamiento y combina sus predicciones para producir una salida final más robusta y precisa que la de un árbol individual.

Para tareas de clasificación, cada árbol vota por una clase y el bosque asigna la clase que recibe la mayoría de los votos (votación dura o hard voting). Opcionalmente, puede usarse votación blanda (soft voting) promediando las probabilidades predichas por cada árbol.

B.2 Fundamentos: ensemble learning y bagging

Random Forest se basa en dos principios fundamentales del aprendizaje por conjuntos:

1. Bagging (Bootstrap Aggregating) [2]:

- Se generan T subconjuntos de entrenamiento a partir del conjunto original mediante muestreo bootstrap: se extraen N muestras con reemplazo (donde N es el tamaño del conjunto original). Esto implica que algunos ejemplos se repiten y otros (~36.8%) quedan fuera (llamados out-of-bag u OOB).
- Cada árbol se entrena con un subconjunto bootstrap diferente.
- La agregación por votación reduce la varianza sin aumentar significativamente el sesgo.

2. Aleatorización adicional en la selección de características:

- A diferencia del bagging estándar (que sólo muestrea filas), Random Forest introduce aleatoriedad también en las

columnas (características).

- En cada nodo de cada árbol, al momento de buscar la mejor división (split), solo se considera un subconjunto aleatorio de m características de las d totales. Típicamente $m = \sqrt{d}$ para clasificación.
- Esta doble aleatorización (filas y columnas) descorrelaciona los árboles, logrando una reducción de varianza aún mayor que el bagging simple.

B.3 Algoritmo de construcción (para clasificación)

El proceso de entrenamiento con T árboles es:

1. Para cada árbol $t=1 \dots T$:

Crear un conjunto bootstrap D_t muestreando con reemplazo N ejemplos del conjunto original.

- Construir un árbol de decisión recursivamente:
- En cada nodo, seleccionar aleatoriamente m características de las d disponibles.
- Entre esas m, elegir la característica y el umbral que maximicen la ganancia de información.
- Dividir el nodo en dos hijos.

Existen otros algoritmos de árboles de decisión que permiten divisiones múltiples.

- Continuar hasta que se alcance un criterio de parada.
 - No se aplica poda (pruning); los árboles se crecen al máximo posible (bajo las restricciones impuestas).
- #### 2. Una vez entrenados los T árboles, para predecir una nueva instancia x:
- Cada árbol genera una predicción de clase y_t
 - El bosque retorna $y = \text{moda}(\{y_1, \dots, y_T\})$

B.4 Hiperparámetros para clasificación

Hiper Parámetro	Notación	Rango típico	Efecto principal	Relevancia para la clasificación
Número de árboles	<code>n_estimators (T)</code>	50-500 (común: 100)	Reduce la varianza y mejora la estabilidad al aumentar T; incrementa el costo computacional	Votores mayor generalización; valores muy altos no mejoran significativamente
Número de características por nodo	<code>max_features (m)</code>	<code>sqrt</code> , <code>log2</code> , <code>0.5</code> o <code>random</code>	Controla la aleatorización; menor m aumenta la diversidad (L, bagging, Random)	Clave para descorrelacionar los árboles; valores muy altos generan correlación
Profundidad máxima	<code>max_depth</code>	None o 10-30	Límite al crecimiento del árbol; menor profundidad reduce sobreajuste pero aumenta sesgo	Puede controlar el sobreajuste; valores altos favorecen la capacidad de generalización
Mínimo de muestras por hoja	<code>min_samples_leaf</code>	1-20 (común: 1, 2, 4)	Regula la complejidad; valores mayores generan árboles más suaves	Útil para evitar árboles muy complejos; valores altos favorecen la generalización
Mínimo de muestras para dividir	<code>min_samples_split</code>	2-40 (común: 2, 5, 10)	Evita divisiones muy específicas; valores mayores generan árboles más suaves	Afecta la complejidad de los árboles; valores altos favorecen la generalización
Criterio de división	<code>criterion</code>	<code>gini</code> , <code>entropy</code>	Define la calidad de la división; Gini es más rápido, entropía más precisa	Diferencias pequeñas; importante mantener consistencia en el criterio
Bootstrap	<code>bootstrap</code>	<code>True</code> , <code>False</code>	Permite muestreo con reemplazo para cada árbol	Desactivarlo elimina la ventaja de bootstrap de bagging
Peso de clases	<code>class_weight</code>	<code>None</code> , <code>balanced</code> , <code>balanced_subsample</code>	Ajusta pesos para manejar desbalanceo de clases	Eficacia en datasets desbalanceados; mejora la generalización
Selección aleatoria	<code>random_state</code>	Entero (ej. 42)	Controla la reproducibilidad del modelo	Importante para reproducibilidad y comparación entre modelos

Tabla 1. Hiperparámetros principales de Random Forest para clasificación Para mejor visualización pg 1.

C. XGBoost(Extreme Gradient Boosting):

C.1 Definición y principio de funcionamiento

XGBoost(Extreme Gradient Boosting) es un algoritmo de aprendizaje supervisado desarrollado por Tianqi Chen [3], basado en la técnica de Gradient Boosting. Pertenecce a la familia de métodos de ensemble learning, específicamente a los métodos de boosting, donde múltiples árboles de decisión se construyen secuencialmente para mejorar progresivamente el rendimiento del modelo.

A diferencia de métodos como Random Forest, donde los árboles se entrenan de manera independiente, en XGBoost cada nuevo árbol aprende de los errores cometidos por los árboles anteriores. El objetivo es minimizar una función de pérdida mediante optimización por gradiente.

Para tareas de clasificación, el modelo genera una puntuación acumulada a partir de todos los árboles y posteriormente aplica una función de decisión para asignar la clase final. En clasificación binaria suele utilizarse la función sigmoide, mientras que en clasificación multiclase se utiliza softmax.

C.2 Fundamentos: Boosting y gradient boosting

XGBoost se basa en dos principios fundamentales:

- El *boosting* es una técnica de aprendizaje por conjuntos donde varios modelos débiles (*weak learners*), generalmente árboles de decisión poco profundos, se combinan secuencialmente para construir un modelo fuerte. Cada nuevo árbol intenta corregir los errores residuales producidos por el conjunto de árboles anteriores. De esta forma, el modelo mejora iterativamente su capacidad predictiva.
- XGBoost implementa una versión optimizada de *Gradient Boosting* [4]. En este enfoque:
 - El modelo inicia con una predicción base.
 - En cada iteración se calcula el error residual respecto a la función de pérdida.
 - Un nuevo árbol se entrena para aproximar dichos errores.
 - Las predicciones del nuevo árbol se agregan al modelo anterior utilizando una tasa de aprendizaje (*learning rate*).

Además, XGBoost incorpora regularización L1 y L2 para controlar el sobreajuste (*overfitting*), así como optimizaciones computacionales que mejoran significativamente la velocidad de entrenamiento.

C.3 Algoritmo de construcción(Para clasificación):

El proceso de entrenamiento de XGBoost consiste en:

Para cada iteración $t=1 \dots T$:

- Calcular las predicciones actuales del modelo.
- Evaluar el error utilizando una función de pérdida.
- Calcular el gradiente y, opcionalmente, la segunda derivada (*Hessian*) de la función de pérdida.
- Construir un nuevo árbol de decisión que minimice el error residual:
 - Evaluar posibles divisiones de los nodos.
 - Seleccionar la división que maximice la ganancia.
 - Aplicar restricciones de profundidad y regularización.
- Agregar el nuevo árbol al modelo utilizando una tasa de aprendizaje.

La predicción final para una nueva instancia x se obtiene sumando las contribuciones de todos los árboles.

C.4 Hiper Parámetros para clasificación

Hiperparámetro	Notación	Rango típico	Efecto principal	Relevancia para la comparación
Número de árboles	<code>n_estimators</code>	100 – 1000	Define cuántos árboles componen el modelo. Más árboles pueden mejorar la precisión, pero aumentan el tiempo de entrenamiento y el riesgo de sobreajuste.	Permite comparar la complejidad y capacidad de aprendizaje frente a Random Forest y CatBoost.
Tasa de aprendizaje	<code>learning_rate (eta)</code>	0.01 – 0.3	Controla la contribución de cada árbol nuevo al modelo final. Valores bajos generan aprendizaje más estable y preciso.	Fundamental para analizar la velocidad de convergencia y estabilidad del entrenamiento respecto a CatBoost.
Profundidad máxima	<code>max_depth</code>	3 – 12	Limita la profundidad de cada árbol. Valores altos permiten capturar relaciones complejas, pero incrementan el sobreajuste.	Permite comparar la complejidad estructural de los árboles entre los distintos algoritmos.
Submuestreo de datos	<code>subsample</code>	0.5 – 1.0	Define la proporción de muestras utilizadas para entrenar cada árbol. Reduce correlación y sobreajuste.	Ayuda a evaluar la capacidad de generalización y robustez del modelo frente a Random Forest.

Tabla 2. Hiper Parámetros principales de XGBoost para clasificación.

Para mejor visualización pg 2. Tablas-Proyecto3-Sistemas.pdf

D. CatBoost:

D.1 Definición y origen:

CatBoost (Categorical Boosting) es un algoritmo de aprendizaje supervisado basado en gradient boosting (impulso por gradiente), desarrollado por Yandex en 2017 [8]. Pertenecce a la familia de métodos de ensemble, específicamente al boosting secuencial, donde los modelos débiles se añaden uno tras otro corrigiendo los errores de los anteriores. Construye árboles en serie, donde cada nuevo árbol se enfoca en los ejemplos que los árboles previos clasificaron incorrectamente.

D.2 Fundamentos de Gradient Boosting

Se basa en el marco de Gradient Boosting Machines (GBM) propuesto por Friedman [3]. En esencia:

1. Se inicializa un modelo base $F_0(X)$ (típicamente una constante igual a la clase mayoritaria o al log-odds).
2. Para cada iteración $t=1, \dots, T$:
 - Se calcula el gradiente negativo de la función de pérdida con respecto a la predicción actual de cada ejemplo. Este gradiente representa el "error residual".
 - Se entrena un árbol de decisión débil para predecir esos gradientes.
 - El árbol se añade al modelo con un factor de aprendizaje (learning rate): $F_t(X)=F_{t-1}(X)+\eta h_t(X)$, con η la tasa de aprendizaje.
3. El modelo final es la suma ponderada de todos los árboles débiles.

D.3 Características distintivas de CatBoost

1. Manejo nativo de características categóricas:
 - Transforma las variables categóricas utilizando estadísticas objetivo (target statistics) basadas en el orden de los datos.
 - Usa una técnica llamada ordered target statistics para evitar el sobreajuste y la fuga de información (evita usar la etiqueta del ejemplo actual para calcular su propia transformación)
 - Permite combinar características categóricas en nuevas características (feature combinations) automáticamente.
2. Ordered boosting (Boosting ordenado):
 - Cada árbol se entrena con gradientes calculados a partir de los mismos datos usados para entrenar árboles anteriores, lo que introduce un sesgo de predicción (sobreajuste por fuga de información).
 - Entrena un conjunto de modelos auxiliares en diferentes permutaciones de los datos. Para cada ejemplo, la predicción del gradiente se calcula usando solo los ejemplos que aparecen antes en la permutación. Esto reduce drásticamente el sobreajuste.
3. Árboles simétricos (oblivious trees): En lugar de árboles de decisión estándar, usa árboles simétricos u "oblivios", donde la misma característica y el mismo umbral se aplican a todos los nodos de una misma profundidad. Esto reduce el sobreajuste, acelera la inferencia y permite implementaciones altamente eficientes

en GPU.

4. Manejo de valores faltantes: Maneja automáticamente valores nulos sin necesidad de imputación previa, tratándolos como una categoría separada en las divisiones.

D.4 Hiper Parámetros de CatBoost

Hiperparámetro	Notación	Rango típico	Efecto principal	Relevancia para la comparación
Número de iteraciones	iterations	100–1000+	Número de árboles construidos secuencialmente	Muy influyente en el rendimiento; demasiadas iteraciones pueden sobreajustar
Tasa de aprendizaje	learning_rate	0.01–0.3 (común: 0.05–0.1)	Controla la contribución de cada árbol	Valores pequeños mejoran generalización pero requieren más iteraciones
Profundidad del árbol	depth	4–10	Define la complejidad de los árboles	Profundidades altas capturan relaciones complejas pero aumentan el riesgo de sobreajuste
Regularización L2	l2_leaf_reg	1–10	Penaliza valores grandes en hojas	Reduce sobreajuste; clave frente a modelos no regularizados
Submuestreo	subsample	0.5–1.0	Proporción de datos usados por iteración	Reduce varianza y mejora eficiencia
Temperatura de bagging	bagging_temperature	0–1	Controla el nivel de aleatorización en el muestreo	Mayor valor → más diversidad; menor riesgo de sobreajuste
Tipo de pérdida	loss_function	Logits, CrossEntropy	Define la función objetivo	Debe ser consistente para comparaciones justas
Manejo de clases	class_weights	None o lista de pesos	Ajusta pesos para clases desbalanceadas	Esencial en datasets desbalanceados
Semilla aleatoria	random_seed	Entero (ej. 42)	Controla la reproducibilidad del modelo	Fundamental para comparaciones consistentes
Manejo de variables categóricas	cat_features	Índice/lista	Especifica variables categóricas	Ventaja clave de CatBoost frente a otros modelos

Tabla 3. Hiper Parámetros principales de CatBoost para clasificación. Para mejor visualización pg 3. Tablas-Proyecto3-Sistemas.pdf

III. Resultados

Para realizar una comparación justa, se utilizó para cada método el mismo código fuente[10], en éste se usaba el 80% del dataset aleatoriamente para el entrenamiento, sólo se fijaron los hiperparámetros que en caso de querer saber sobre la elección de los mismos, se puede consultar el mismo link de las tablas. Ahora, sobre el dataset, también puede ser encontrado en la referencia 10, es un dataset del icfes donde recopila datos no sólo de puntajes por área si no socioeconómicos, tenía muchas variables que no utilizamos para el entrenamiento así que nos quedamos con unas cuantas que podrán verse en el dataset el github, esta depuración se hizo con el objetivo de eliminar ruido y hacer menos pesado el procesamiento del dataset.

Para la comparación usamos los valores mencionados en la introducción, las tablas de estos valores se encontrarán [aquí](#); y en el presente informe sólo daremos los resultados generales por facilidad.

El objetivo consistió en clasificar a los estudiantes en cinco categorías quintiles de desempeño académico (para entender el porqué se decidió por 5 categorías ver el documento de tablas) a partir de variables socioeconómicas, familiares, institucionales y de hábitos de estudio.

Los resultados obtenidos muestran que CatBoost fue el modelo con mejor desempeño global en prácticamente todas las métricas evaluadas.

Estos resultados indican que CatBoost logró una mayor capacidad de discriminación entre categorías, una menor cantidad de errores graves y una mejor capacidad de generalización sobre datos no vistos. La superioridad de CatBoost era esperable debido a que el dataset contiene una gran cantidad de variables socioeconómicas y características institucionales del colegio, así como relaciones no lineales complejas entre

estos factores y el desempeño académico. Este algoritmo está especialmente diseñado para trabajar con datos tabulares heterogéneos y variables categóricas de alta cardinalidad.

Por otro lado, Random Forest obtuvo un desempeño intermedio. El modelo alcanzó una exactitud de 36.07%, aunque presentó un F1-score menor (0.3281), indicando una menor capacidad para equilibrar correctamente la clasificación entre las diferentes categorías. Asimismo, el valor ROC-AUC fue de 0.6917, mostrando una capacidad de discriminación aceptable, aunque inferior a la de CatBoost.

En cuanto a los errores, Random Forest presentó un MAE de 1.0949 y un RMSE de 1.5380, siendo este último el valor más alto entre los modelos evaluados. Esto indica una mayor presencia de errores severos y una mayor confusión entre categorías extremas.

Finalmente, XGBoost presentó un desempeño muy cercano al de CatBoost. Este algoritmo obtuvo una exactitud de 38.01%, un F1-score de 0.3666 y un valor ROC-AUC de 0.7119, lo que evidencia una capacidad de clasificación muy similar a la obtenida por CatBoost.

Este comportamiento puede explicarse porque XGBoost utiliza una estrategia de boosting secuencial que permite corregir iterativamente los errores cometidos por árboles anteriores, mejorando progresivamente la capacidad predictiva del modelo. Aunque el modelo presentó un RMSE de 1.3747, ligeramente superior al de CatBoost, sus métricas globales de clasificación continúan siendo altamente competitivas.

El valor ROC-AUC permite medir la capacidad del modelo para separar correctamente las clases. CatBoost presentó la mejor capacidad discriminativa (0.7128), XGBoost mostró un desempeño prácticamente equivalente (0.7119) y Random Forest presentó la menor capacidad de separación entre categorías (0.6917).

El MAE representa el error absoluto promedio entre la categoría real y la predicción. CatBoost obtuvo el menor error promedio (0.9636), indicando que sus predicciones se encuentran más cercanas a la categoría correcta. XGBoost presentó un error ligeramente superior (0.9696), mientras que Random Forest alcanzó el valor más alto (1.0949), reflejando una mayor desviación respecto a las clases reales.

Por su parte, el RMSE penaliza con mayor intensidad los errores grandes. CatBoost volvió a presentar el menor valor (1.3675), lo que demuestra una menor cantidad de errores severos. Random Forest obtuvo el mayor RMSE (1.5380), mientras que XGBoost alcanzó un valor intermedio (1.3747).

En términos computacionales, XGBoost fue significativamente más rápido, con un tiempo de entrenamiento de 9650.05 segundos, debido a la eficiencia de sus mecanismos de optimización y paralelización. Sin embargo, esta velocidad estuvo acompañada de un desempeño ligeramente inferior al de CatBoost. Por el contrario, CatBoost presentó el mayor tiempo de entrenamiento (27093.61 segundos) debido al entrenamiento secuencial característico de los métodos boosting y al procesamiento avanzado de variables

categorías. No obstante, este mayor costo computacional produjo el mejor rendimiento global. Random Forest mostró un tiempo intermedio (22332.14 segundos), ofreciendo un equilibrio moderado entre desempeño y eficiencia computacional.

La matriz de confusión de CatBoost muestra una mayor capacidad para identificar correctamente estudiantes de desempeño bajo y alto, además de un mejor equilibrio general entre clases y una menor cantidad de errores extremos.

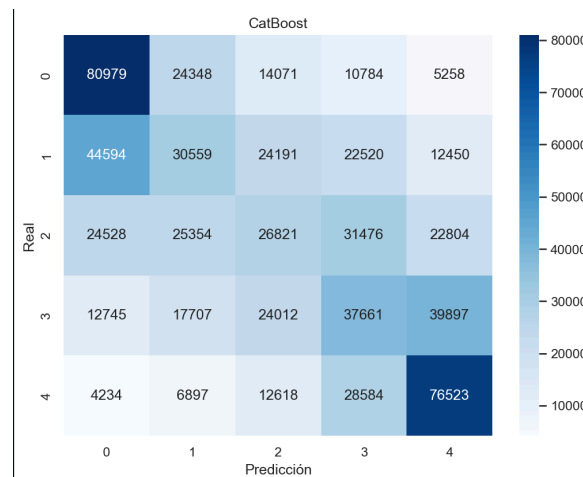


Imagen 1. Matriz de confusión para la técnica CatBoost

Se observa que la categoría media continúa siendo la más difícil de clasificar, ya que muchos estudiantes de desempeño medio son confundidos tanto con categorías bajas como altas. Esto sugiere que las características socioeconómicas y escolares del grupo medio presentan un mayor nivel de solapamiento.

Por otro lado, la matriz de confusión de Random Forest evidencia una buena identificación de estudiantes de bajo desempeño, un rendimiento aceptable en estudiantes de alto desempeño y mayores dificultades para clasificar correctamente la categoría media. El modelo parece favorecer las clases extremas, especialmente la clase baja, posiblemente debido a patrones socioeconómicos más diferenciados en dichos grupos.



Imagen 2. Matriz de confusión de Random Forest

Finalmente, la matriz de confusión de XGBoost presenta una capacidad de clasificación muy cercana a

la observada en CatBoost, mostrando una adecuada separación entre categorías y una menor dispersión de errores en comparación con Random Forest. Sin embargo, persisten dificultades para clasificar correctamente los estudiantes ubicados en los quintiles intermedios, donde existe una mayor heterogeneidad de características.

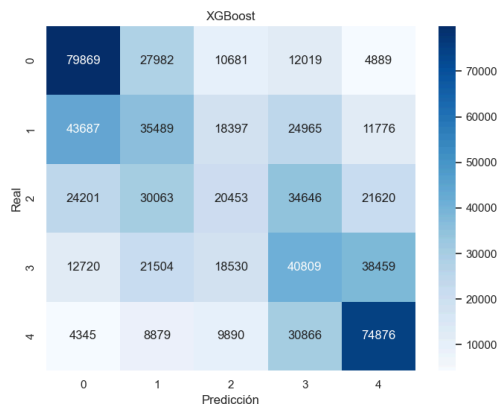


Imagen 3. Matriz de confusión para la técnica XGBoost

Los resultados obtenidos también permiten extraer conclusiones importantes sobre el dataset ICFES. Las variables familiares, económicas y escolares demostraron tener una fuerte relación con el desempeño académico, aportando información altamente discriminadora para los modelos. La capacidad predictiva alcanzada evidencia que el rendimiento académico está significativamente condicionado por factores socioeconómicos e institucionales.

Asimismo, las variables relacionadas con el colegio mostraron una influencia considerable sobre el desempeño de los estudiantes. Los resultados sugieren la existencia de diferencias estructurales importantes entre instituciones educativas que impactan directamente los resultados académicos.

Todos los modelos presentaron dificultades para clasificar correctamente la categoría media. Esto indica que los estudiantes de rendimiento intermedio comparten características con ambos extremos, existe una mayor heterogeneidad interna y los límites entre categorías no son completamente lineales.

IV. Conclusiones

Los resultados demuestran que CatBoost es la técnica más adecuada para este problema de clasificación de quintiles de desempeño académico, debido a su capacidad para manejar variables categóricas, modelar relaciones no lineales complejas y reducir los errores de clasificación. El modelo obtuvo los mejores resultados en todas las métricas evaluadas, destacándose especialmente en Accuracy, F1-score, ROC-AUC y reducción de errores.

XGBoost presentó un desempeño prácticamente equivalente al de CatBoost, obteniendo diferencias mínimas en las métricas de clasificación y mostrando además el menor tiempo de entrenamiento. Esto lo convierte en una alternativa altamente competitiva cuando se busca un equilibrio entre precisión predictiva y eficiencia computacional.

Por su parte, Random Forest alcanzó resultados inferiores, aunque mantuvo un comportamiento estable y una capacidad de discriminación aceptable. Sin embargo, su desempeño fue superado consistentemente por los métodos de boosting, lo que evidencia las ventajas de estos algoritmos para capturar patrones complejos presentes en datos educativos.

Los resultados obtenidos permiten concluir que las variables socioeconómicas de los estudiantes y las características institucionales de los colegios poseen una relación significativa con el desempeño académico en las pruebas Saber 11. La capacidad predictiva alcanzada por los modelos confirma que factores asociados al contexto familiar, económico y escolar influyen de manera importante en los resultados obtenidos por los estudiantes.

V. Referencias

- [1] T. M. Mitchell, Machine Learning, 1st ed. New York, NY, USA: McGraw-Hill, 1997.
- [2] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling modern machine learning practice and the classical bias-variance trade-off," Proc. Nat. Acad. Sci. USA, vol. 116, no. 32, pp. 15849–15854, Aug. 2019.
- [3] C. M. Bishop and H. Bishop, Deep Learning: Foundations and Concepts, 1st ed. Cham, Switzerland: Springer, 2023.
- [4] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [5] L. Breiman, "Bagging predictors," Mach. Learn., vol. 24, no. 2, pp. 123–140, Aug. 1996.
- [6] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [7] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.
- [8] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [9] T. Chen, "Xgboost: Extreme Gradient Boosting," R package version 0.4-2, 2015.
- [10] XGBoost Developers, "XGBoost Documentation," Distributed Machine Learning Community (DMLC). [Online]. Available: <https://dmlc.cs.washington.edu/xgboost.html>
- [11] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2017.
- [12] E. Fix and J. L. Hodges, "Discriminatory analysis - nonparametric discrimination: Consistency properties," USAF School of Aviation Medicine, Randolph Field, Texas, Project Rep. 21-49-004, 1951. Republicado en Int. Statist. Rev., vol. 57, no. 3, pp. 238–247, 1989.
- [8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2018, pp. 6638–6648.
- [9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, Oct. 2001.
- [10] A. D Gomez P, L. Y Quiñones G, "código fuente de técnicas de clasificación" Abril 2026, https://github.com/Angomezp/Sistemas_Inteligentes/tree/Dev-Cuadrilo/Proyecto